

Vibe Coding and Data Analysis

One toy example: a linear equation in two variables Project: Titanic

AI in data analysis

- AI can help you get answers **faster**. That part is easy.
- The hard part is getting answers that are **correct**.
- You need to **break problems into small steps** and **ask clear questions**.
- Today: one running example—a **linear equation with two inputs** (x_1, x_2)—then the real **Titanic** dataset.

Our toy model: linear in two variables (same story all lesson)

We use one simple formula as a “model” (first degree in x_1 and x_2):

$$y = ax_1 + bx_2 + c$$

- x_1, x_2 : two inputs (two features, two numbers per row)
- y : output (one number)
- a, b, c : unknown—we **learn** them from data

For **train / validation / test**: we get rows (x_1, x_2, y) , fit a, b, c , then check on **new** rows we did not use for training.

Concrete scenario: prep and a short quiz

Imagine a class before a **small math quiz**. Each row is **one student**.

- x_1 : Did the student finish the **practice packet**? (1 = yes, 0 = no)
- x_2 : Did they attend **peer tutoring** that week? (1 = yes, 0 = no)
- y : **Quiz score** on a simple scale (made-up numbers for class—real life would be messier).

We pretend there is a hidden linear rule linking prep choices to the score. Your job in the toy exercise is to **recover** that rule from the table, like fitting a line in math—but with **two** yes/no inputs instead of one x on an axis.

Dataset basics: a few words

Sample One row, e.g. one triple (x_1, x_2, y) .

Feature The inputs used to predict y ; here, x_1 and x_2 .

Label The y we want to learn or predict.

Layout Many samples in a **table**: one row per sample.

What does data look like?

- **Table:** numbers and text, one row per sample. (We use this most today.)
- **Multimodal:** images, audio, video too—different inputs, same idea: input $\rightarrow$ model $\rightarrow$ output.

x_1	x_2	y
0	0	1
1	0	3
0	1	2

Same quiz story: each row is one student; x_1, x_2 are features; y is the label (score).

Split the data three ways

- **Training set:** used to **learn** a, b, c (or any model).
- **Validation set:** used to **tune** choices (which formula, which features), **not** the final grade.
- **Test set:** used **last**, like a **final exam**—only rows the model has not seen for training.

Why split? If you train and “grade” on the same rows, scores look **too good** and often fail on new data.

What is a model? (again, linear in two variables)

- **Input:** x_1 and x_2
- **Output:** $\hat{y} = ax_1 + bx_2 + c$ (hat means “prediction”)
- **Model:** the rule from (x_1, x_2) to $\hat{y}$; here, one **linear** equation in two variables (plus intercept c).

Example (teacher “answer key” for the toy): $y = 2x_1 + x_2 + 1$. Students only see the table and try to recover a, b, c .

Toy data: four students

Same scenario: four students, four combinations of packet / tutoring. True rule (for demo only): $y = 2x_1 + x_2 + 1$.

id	x_1	x_2	y
1	0	0	1
2	1	0	3
3	0	1	2
4	1	1	4

Training: use only some rows to learn a, b, c . Testing: use (x_1, x_2) pairs **not** in training and compare $\hat{y}$ to true y .

Same idea as “different lines” in algebra

- With **one** input x , a linear model $\hat{y} = mx + b$ is a **line** in the (x, y) plane—you try different slopes m and intercepts b .
- With **two** inputs, $\hat{y} = ax_1 + bx_2 + c$ picks a **plane** in (x_1, x_2, y) -space. Different (a, b, c) mean different “tilts”—like different lines, but one dimension up.
- We do not draw the plane in class; we only **plug in** (x_1, x_2) and compare $\hat{y}$ to real y .

Try three linear guesses on the same four points

True scores y are fixed. We try three formulas (three choices of a, b, c):

id	x_1	x_2	y	$\hat{y} = x_1 + x_2$	$\hat{y} = 2x_1 + x_2$	$\hat{y} = 2x_1 + x_2 + 1$
1	0	0	1	0	0	1
2	1	0	3	1	2	3
3	0	1	2	1	1	2
4	1	1	4	2	3	4

First guess: no intercept, weak weights—**every row is off**. Second: right slopes but **forgets** $+1$ —still wrong on every row. Third: **matches every** y on this tiny clean table. The Python demo uses **many** rows with small random noise so scores do not fit any line exactly, but the **same** three formulas still compete.

Which guess is “closest”? Introducing SSE

Residual for row i : $e_i = y_i - \hat{y}_i$ (actual minus prediction).

SSE (sum of squared errors) adds up squared residuals:

$$\text{SSE} = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

We **square** so a miss of 2 counts more than two misses of 1, and sign (+/-) does not cancel out.

Pencil example ($n = 4$, exact table above):

Model	SSE	Verdict
$\hat{y} = x_1 + x_2$	10	poor
$\hat{y} = 2x_1 + x_2$	4	better
$\hat{y} = 2x_1 + x_2 + 1$	0	best on this noise-free table

With noisy real data, even the “right” rule usually has $\text{SSE} > 0$; we still pick the (a, b, c) that makes SSE **smallest** on the training set (same idea as least squares).

Where do 10 and 4 come from? (step-by-step)

Four rows: $(x_1, x_2, y) = (0, 0, 1), (1, 0, 3), (0, 1, 2), (1, 1, 4)$. Compute $\hat{y}_i$, residual $e_i = y_i - \hat{y}_i$, square, then add.

(1) $\hat{y} = x_1 + x_2$

row i	y_i	$\hat{y}_i$	$e_i = y_i - \hat{y}_i$	e_i^2
1	1	0	1	1
2	3	1	2	4
3	2	1	1	1
4	4	2	2	4
SSE =				1+4+1+4 = 10

(2) $\hat{y} = 2x_1 + x_2$

row i	y_i	$\hat{y}_i$	e_i	e_i^2
1	1	0	1	1
2	3	2	1	1
3	2	1	1	1
4	4	3	1	1
SSE =				1+1+1+1 = 4

Where does 0 come from? (perfect fit on this table)

(3) $\hat{y} = 2x_1 + x_2 + 1$ matches the hidden rule used to build the toy table.

row i	y_i	$\hat{y}_i$	e_i	e_i^2
1	1	1	0	0
2	3	3	0	0
3	2	2	0	0
4	4	4	0	0
SSE =				0

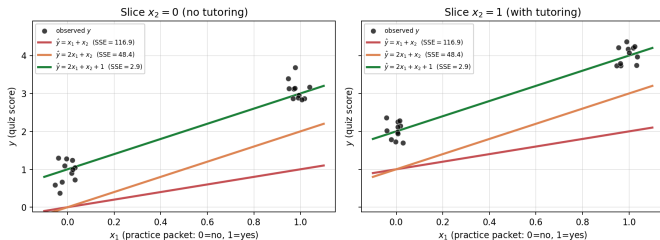
Every residual is zero, so every squared residual is zero—only happens here because the table was built from this exact linear rule with no noise.

Training = search for a small SSE

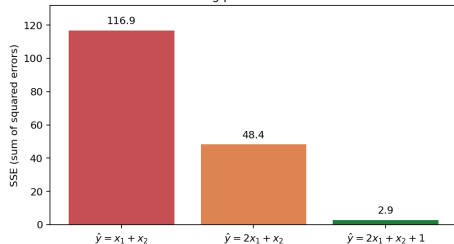
- **Training**: try many (a, b, c) (grid, calculus, or code) and choose the one with **lowest SSE** on training rows.
- Other losses exist (absolute errors, robust losses, ...); SSE is the standard starting point for linear regression.
- **Validation / test** use the same idea on **other** rows you did not use to pick a, b, c —otherwise the score is too optimistic.

Python: points in the plane + model as a line

Synthetic scores: $y = 2x_1 + x_2 + 1 + \varepsilon$, $\varepsilon \sim \mathcal{N}(0, 0.32^2)$, 44 students. Green line tracks the hidden rule best; SSE still ranks models.



Total error on all training points — smallest wins for this loss



$$SSE = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

y_i = actual score, $\hat{y}_i$ = model prediction for row i .
Square each residual so large errors weigh more; add over all students.

Many students per (x_1, x_2) with small random noise (scores not exactly on a line). Black dots = observed y ; colored lines = three formulas. The **green** line ($\hat{y} = 2x_1 + x_2 + 1$) hugs the cloud best; SSE bar (with formula in the figure) ranks total error. Also: `toy_model_slice_grid.png`, `toy_corner_groups.png`.

A simple workflow (still using the linear toy)

- 1 **State the question:** what do we predict? (Here: quiz score from packet + tutoring flags.)
- 2 **Clean the table:** drop columns that do not belong; do not feed useless features.
- 3 **Look and plot:** does y change in a simple way when x_1 or x_2 changes? (For this toy, the truth is linear.)
- 4 **Missing values:** why they hurt (less info, bias). Simple fixes: drop rows, or fill with a typical value—say when each is OK.
- 5 **Split** → **build model** → **train / validate**.
- 6 **Test:** new rows; explain why “exam questions you already practiced” is not a fair final score.

Real data: Titanic

- Each row: one passenger. Features: sex, cabin class, age, ... Label: survived or not.
- Same steps: question, clean, explore, missing values, split, model, then test.
- With AI: **check** column names, row counts, and splits yourself—do not trust a paragraph of text alone.

Questions you can study

- ① Which features matter most for survival? (“Important” in a model $\neq$ cause.)
- ② Is the survival gap between sexes large? Is it **statistically significant**? (Needs a clear test or comparison.)
- ③ Does the data support “women and children first”? (Define “child”; remember confounders like class.)

Assignment: short PDF report

- What the data looks like (rows, columns)
- What model you used and how you split the data
- Main results (figure or table)
- Short conclusion: answer one or more questions above

Say what the AI helped with and what **you** verified (code, counts, plots).